

Diseño de Arquitectura de Microservicios mediante C4 Nivel 3: TiendaTech

Farinango Guandinango Freddy Vladimir Gaibor Rodriguez Jeremy Ruperto Villamarin Cuenca Ivan Andres
Aplicaciones Distribuidas Aplicaciones Distribuidas Aplicaciones Distribuidas

Resumen—Este trabajo presenta el diseño de la arquitectura de microservicios del sistema TiendaTech empleando el modelo de diagramas C4 en su nivel 3 (componentes). Se identifican las responsabilidades de cada microservicio, los mecanismos de comunicación síncrona y asíncrona, así como las tecnologías necesarias para la integración y operación del sistema distribuido. El diseño propone una separación de responsabilidades que facilita la escalabilidad y el mantenimiento, combinando REST y RabbitMQ para equilibrar la rapidez de respuesta con el desacoplamiento entre servicios.

Index Terms—Microservicios, C4 Nivel 3, API Gateway, RabbitMQ, REST, PostgreSQL, arquitectura distribuida.

I. INTRODUCCIÓN

El diseño de sistemas distribuidos mediante arquitecturas de microservicios ha cobrado gran relevancia en el desarrollo de aplicaciones modernas. Este paradigma propone descomponer una aplicación monolítica en servicios pequeños, autónomos e independientemente desplegables, cada uno con una responsabilidad bien definida [1].

El presente trabajo describe la arquitectura del sistema TiendaTech, una plataforma de comercio electrónico diseñada bajo el enfoque de microservicios. Para su modelado se utiliza el diagrama C4 en su nivel 3, que describe los componentes internos de los contenedores del sistema y sus interacciones.

El repositorio del proyecto se encuentra disponible en: <https://github.com/ivillamarinc/aplicaciones-distribuidas>

II. OBJETIVO

Diseñar la arquitectura de microservicios del sistema TiendaTech mediante un diagrama C4 nivel 3, identificando responsabilidades, mecanismos de comunicación síncronos y asíncronos, así como las tecnologías necesarias para la integración y operación del sistema distribuido.

III. DESCRIPCIÓN DE LA ARQUITECTURA

La arquitectura propuesta sigue el principio de responsabilidad única (SRP) y está compuesta por los siguientes elementos principales:

III-A. Frontend Web TiendaTech

Interfaz desarrollada con HTML, CSS y JavaScript que permite al usuario interactuar con el sistema. Envía todas las solicitudes al API Gateway como punto de entrada centralizado.

III-B. API Gateway

Implementado con Spring Boot y Spring Cloud Gateway, constituye el único punto de acceso al sistema. Gestiona la autenticación, autorización y el enrutamiento de solicitudes hacia los microservicios correspondientes.

III-C. Microservicio de Usuarios

Construido con Spring Boot, REST y JWT. Gestiona el registro de usuarios, inicio de sesión, autenticación JWT y recuperación de credenciales. Dispone de su propia base de datos PostgreSQL denominada *BD Usuarios*.

III-D. Microservicio de Productos

Implementado con Spring Boot y REST. Administra el catálogo de productos, incluyendo categorías, imágenes, búsquedas, precios, detalles y control de inventario disponible. Persiste su información en la *BD Productos*.

III-E. Microservicio de Carrito

Desarrollado con Spring Boot y REST. Gestiona los productos agregados al carrito, actualiza información de productos y calcula los totales previos a la compra. Utiliza la *BD Carrito* para almacenar los estados activos de compra.

III-F. Microservicio de Pedidos y Facturación

Construido con Spring Boot y REST. Gestiona la creación de pedidos, facturas y el historial de compras. Publica eventos hacia la cola de mensajes indicando que un pedido ha sido creado. Consume eventos de pago para actualizar el estado del pedido. Se apoya en la *BD Pedidos y Facturación*.

III-G. Microservicio de Pagos

Implementado con Spring Boot, REST y la API de PayPal. Procesa los pagos utilizando PayPal, obtiene el estado de disponibilidad y registra el estado de cada transacción. Publica eventos de pago confirmado o rechazado hacia la cola de mensajes. Utiliza la *BD Pagos*.

III-H. Cola de Mensajes

Implementada con RabbitMQ. Distribuye y enruta eventos entre microservicios de forma asíncrona sin necesidad de esperar una respuesta inmediata, garantizando el desacoplamiento entre los servicios productores y consumidores.

IV. COMUNICACIÓN Y TECNOLOGÍAS

IV-A. Comunicación Síncrona

La comunicación síncrona se realiza mediante el protocolo HTTP REST con intercambio de datos en formato JSON. Este mecanismo se emplea para las operaciones que requieren una respuesta inmediata, tales como:

- Solicitudes del Frontend al API Gateway.
- Enrutamiento del API Gateway hacia los microservicios de Usuarios, Productos y Carrito.
- Registro de pedidos y facturas desde el API Gateway hacia el microservicio de Pedidos y Facturación.
- Procesamiento de pagos desde el API Gateway hacia el microservicio de Pagos.

IV-B. Comunicación Asíncrona

La comunicación asíncrona se implementa mediante RabbitMQ. Este mecanismo permite el intercambio de eventos sin bloquear el flujo principal de ejecución, siendo utilizado en los siguientes escenarios:

- Publicación del evento *pedido_creado* por parte del microservicio de Pedidos y Facturación.
- Consumo del evento de pago por parte del microservicio de Pedidos para actualizar el estado del pedido.
- Publicación del evento de pago confirmado o rechazado por parte del microservicio de Pagos.

IV-C. Tecnologías Utilizadas

Las tecnologías empleadas en la arquitectura se resumen a continuación:

- **Spring Boot / Spring Cloud Gateway:** desarrollo de microservicios y API Gateway.
- **REST / JSON:** comunicación síncrona entre componentes.
- **JWT:** autenticación y autorización de usuarios.
- **RabbitMQ:** mensajería asíncrona basada en eventos.
- **PostgreSQL:** almacenamiento independiente para cada microservicio.
- **PayPal API:** procesamiento de pagos externos.
- **HTML, CSS, JavaScript:** desarrollo del Frontend Web.

V. RETROALIMENTACIÓN Y MEJORAS

Durante la revisión del diseño inicial se identificó la ausencia de mecanismos de comunicación asíncrona y la falta de diferenciación explícita entre los tipos de interacción entre servicios. Como resultado de esta retroalimentación se realizaron las siguientes mejoras:

1. Incorporación de RabbitMQ como componente de mensajería asíncrona basada en eventos.
2. Identificación y etiquetado explícito de las comunicaciones síncronas (*sync*) y asíncronas (*async*) dentro del diagrama C4.
3. Definición clara de los eventos publicados y consumidos por cada microservicio.

Estas mejoras contribuyen a un diseño más robusto, flexible y alineado con las buenas prácticas de arquitecturas orientadas a eventos.

VI. ANÁLISIS Y DISCUSIÓN

La arquitectura propuesta sigue los principios fundamentales del diseño de microservicios [?]: separación de responsabilidades, independencia de bases de datos por servicio y comunicación bien definida entre componentes.

El uso del patrón API Gateway centraliza la autenticación y el enrutamiento, simplificando la interacción del Frontend con el sistema sin necesidad de conocer la topología interna de los servicios [3].

La combinación de REST para operaciones síncronas y RabbitMQ para comunicación asíncrona permite equilibrar la latencia de respuesta en operaciones críticas (consulta de productos, autenticación) con la desconexión temporal requerida en flujos de mayor complejidad (creación de pedidos, notificación de pagos).

Sin embargo, la arquitectura distribuida incrementa la complejidad operativa al requerir mecanismos adicionales de coordinación, monitoreo, gestión de fallos y consistencia eventual entre los microservicios.

VII. CONCLUSIÓN

La arquitectura de microservicios diseñada para TiendaTech organiza el sistema en servicios independientes que facilitan la escalabilidad y el mantenimiento de la plataforma. La separación de responsabilidades mejora la flexibilidad del sistema, aunque también incrementa la complejidad operativa al requerir mecanismos adicionales de coordinación, monitoreo y gestión de comunicaciones.

El uso combinado de REST y RabbitMQ resulta necesario y adecuado para equilibrar la rapidez de respuesta con el desacoplamiento entre los microservicios. La adopción del modelo C4 nivel 3 como herramienta de documentación arquitectónica facilita la comprensión del sistema por parte de todos los involucrados en su desarrollo y mantenimiento.

REFERENCIAS

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015.
- [2] M. Fowler y J. Lewis, "Microservices: a definition of this new architectural term," *martinfowler.com*, Mar. 2014. [En línea]. Disponible en: <https://martinfowler.com/articles/microservices.html>
- [3] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [4] S. Brown, *The C4 Model for Visualising Software Architecture*. Leanpub, 2018.
- [5] IBM Cloud Education, "What is RabbitMQ?," *IBM Cloud Learn Hub*, 2020. [En línea]. Disponible en: <https://www.ibm.com/topics/rabbitmq>

APÉNDICE A

DIAGRAMA DE COMPONENTES C4 NIVEL 3 — TIENDATECH

La Figura 1 presenta la arquitectura completa de microservicios del sistema TiendaTech modelada mediante el diagrama C4 nivel 3. El diagrama muestra los contenedores principales del sistema, sus componentes internos, las bases de datos independientes de cada microservicio, la cola de mensajes RabbitMQ y los mecanismos de comunicación síncrona (HTTP REST/JSON) y asíncrona (RabbitMQ) entre ellos.

**Arquitectura de Microservicios -
TiendaTech**
Diagrama de contenedores para el diseño de
microservicios del PFC.

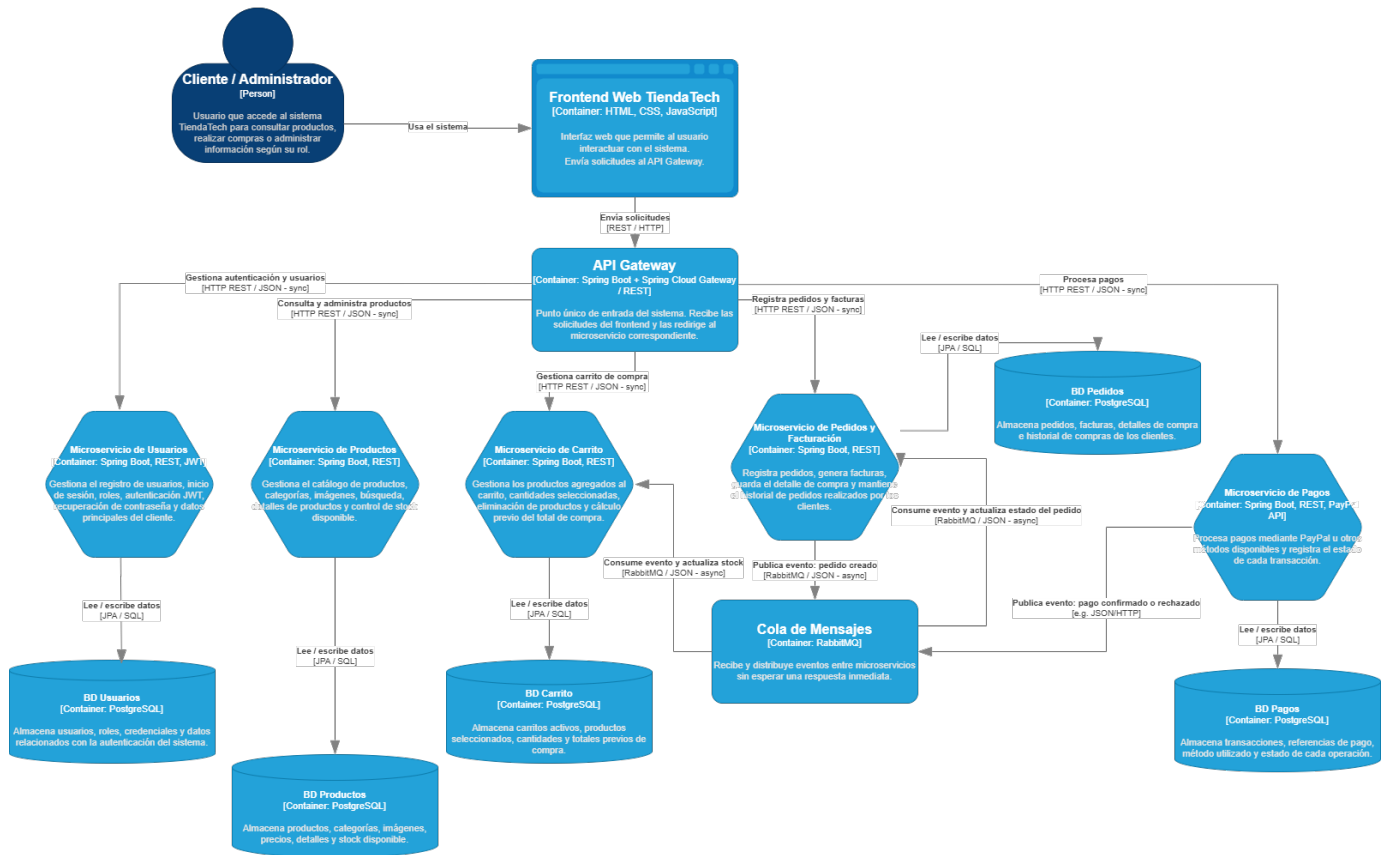


Figura 1. Arquitectura de Microservicios de TiendaTech — Diagrama C4 Nivel 3.